Iteration 4 — Change Report

Ten bags, a corner that is 1.25 m wide, and a recovery whose every rung failed

Summary

This was much the best iteration so far, and most of what follows is about the two places where it still was not good enough. Neither of them is a tuning problem, and that is the main thing this report has to say.

The clearance. Every autonomous run came within 2.5–5.6 cm of a lethal cell at least once. That is not the controller losing the path — the robot tracked its plan to a median of 3–10 mm all iteration. **The plan itself goes there.** Through the hard corner the published plan keeps its centre line 0.48–0.52 m from the nearest lethal cell and puts the robot's body on top of one, and it does that because every obstacle term in this stack — the planner's `cost_penalty`, and MPPI's `CostCritic` — scores the **centre cell only**. The footprint is used for a yes/no collision veto and never for a gradient. For a robot whose inscribed radius is 0.22 m but whose flank is 0.49 m out, half a metre of centre clearance *is* zero flank clearance. Four controlled experiments against the real cost map — smoother off, `cost_penalty` 3 → 8 → 15, `cost_scaling_factor` 1.7 → 1.0, and two new lattices — moved that minimum by nothing at all.

Why finish → start was harder, and why that turn. That corner is the narrowest point of the course: 1.25 m, as mapped, against a robot 0.98 m wide and 1.08 m long. Because those two numbers are nearly equal, the width the footprint needs grows steeply with how obliquely it is presented — 0.98 m square-on, 1.14 m at 20°, and 1.25 m at 30°, at which point it exactly fills the gap. Going finish → start the route turns 20–26° *inside* that gap; going start → finish it crosses the same gap turning 4°. That is the whole of the asymmetry, and it is geometry, not tuning.

The recovery. Iteration 3's gate fix works — runs 3, 4 and 10 all entered the round robin, which is what last iteration's run 8 could not do. Then every rung below the gate failed: **eight BackUp attempts, eight aborts** with error 714 “Collision Ahead”, four of them before the robot moved at all. So the round robin reached `ClearEntireCostmap` in 5.1–8.1 s, and twice went on to clear *both* maps. Clearing is supposed to be the last resort; it was in practice the first rung that did anything. That is the one thing this iteration actually fixes.

The one-line version

The course now has a 1.25 m pinch and a robot that needs 1.25 m to turn in it, so the clearance is at its geometric floor and no parameter will lift it. Fix the recovery instead, and fix the 6% yaw-scale error that is painting every obstacle 5–14 cm too fat — that is where the missing centimetres actually are.

1. Did the IMU latency changes work?

Half of them, and the half that landed did exactly what was predicted.

What was actually running

The FTDI `latency_timer` fix is in and live: `/sys/bus/usb-serial/devices/ttyUSB1/latency_timer` now reads **1**, and `/etc/udev/rules.d/99-um7-latency.rules` is in place, so it survives a replug.

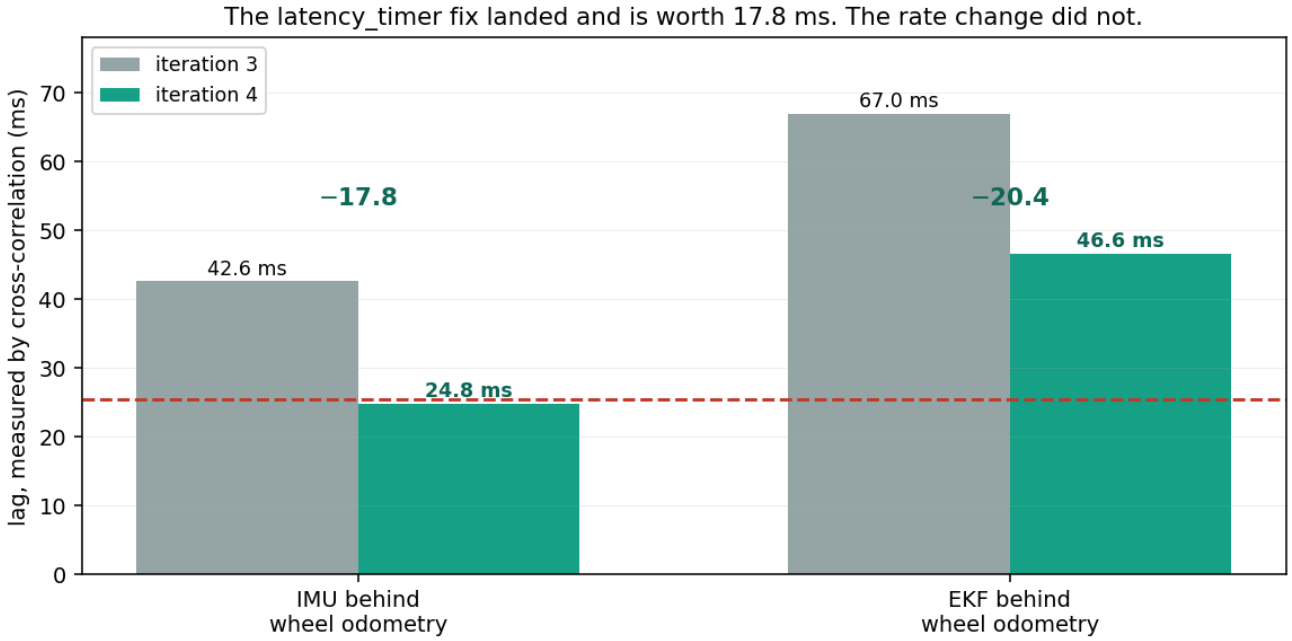
The rate change is **not** in. The IMU published at 19.55–19.62 Hz in all ten bags — the driver's 20 Hz default, not the 60 Hz the launch file now asks for. The reason is that the edit never got built: the source file carries the change, but the copy `ros2 launch` actually reads still has none.

```
src/imprimis_hardware_platform/launch/imprimis_hardware.launch.py
  parameters=[{"port": "/dev/ttyUSB1", "update_rate": 60}]  (edited 18:59)
```

```
install/imprimis_hardware_platform/share/.../imprimis_hardware.launch.py
  parameters=[{"port": "/dev/ttyUSB1"}]  (built 15:04)
```

A `colcon build --packages-select imprimis_hardware_platform` and a relaunch is all it needs.

What the bags measured



Lag of each signal behind the wheel odometry, by cross-correlation on header stamps, pooled over all ten bags. The estimator is the one from latency_claude_3 and its sign convention was re-checked against a synthetic 30 ms delay.

signal	latency_claude_3	iteration 4	change
IMU behind wheel odometry	42.6 ms	**24.8 ± 4.8 ms**	–17.8 ms
EKF `/odometry/filtered/local` behind wheel odometry	67 ms	**46.6 ± 5.0 ms**	–20.4 ms
published odom behind raw wheels	10.0 ms	10.0 ms (unchanged, as predicted)	—

The predicted saving from `latency_timer` 16 → 1 was about 15 ms. The measured saving is 17.8 ms. And the residual accounts for itself exactly: 24.8 ms of remaining lag against a mean sample age of **25.5 ms** at the 19.6 Hz the driver is still running at. In other words **everything that is left is the rate**, which is the half that was not built.

What building it is worth

At 60 Hz the mean sample age falls from 25.5 ms to 8.3 ms, so expect the IMU to sit about **8–10 ms** behind the wheels instead of 24.8, and the EKF to come in around 30 ms instead of 46.6. That is worth having, but note what it is not: it is about 17 ms on a loop whose dominant term is a 190 ms motor time constant. The latency campaign is close to finished.

A correction to iteration 3

Iteration 3 reported “280–320 ms of dead time between a yaw command and the robot responding”. That number is right but the label was wrong, and it matters because dead time and lag want different fixes. Measured here, command to EKF-measured yaw rate is 250–300 ms in every autonomous run, and it decomposes as about **45 ms of real dead time** (the step tests in latency_claude_3), **~190 ms of first-order motor lag** (the same tests' time constant) and **47 ms of EKF reporting delay**. Only the 45 ms is transport. The rest is the machine accelerating, and no amount of serial-link work will remove it — it is a PID and inertia question.

2. Why finish → start was harder, and why that turn

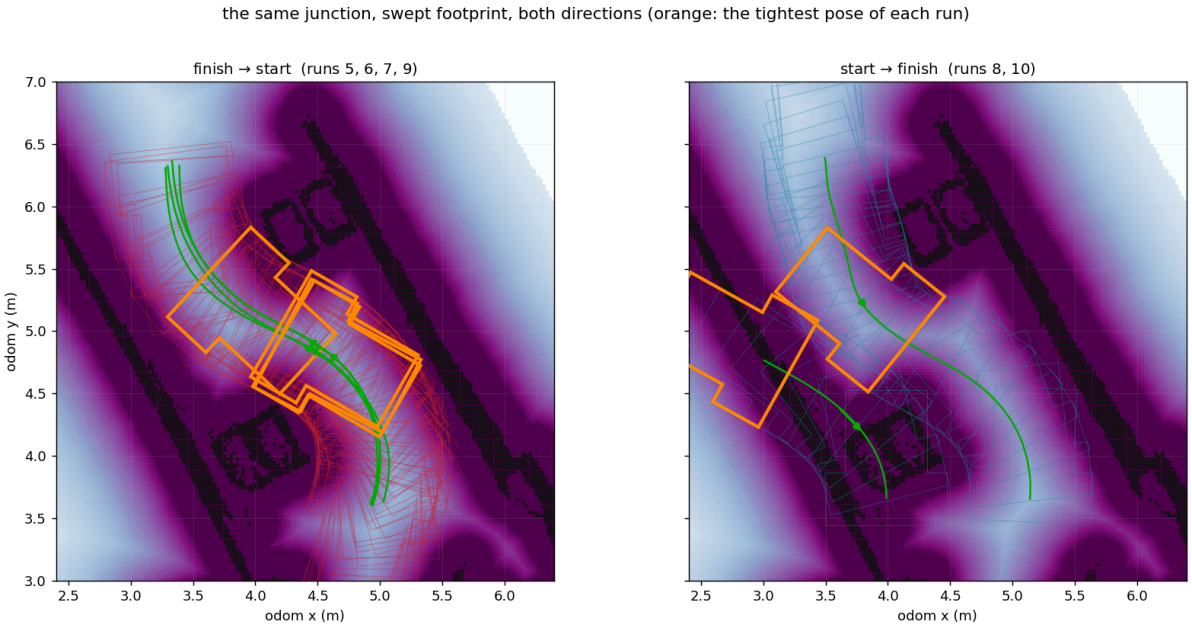
Bags 5 to 10 share one odometry frame — `/odometry/filtered/local` starts at the origin in bag 5 and the stack was never restarted after it — so those six runs can be laid on one picture and compared directly. Bags 1 to 4 share an earlier frame.

Where it happened

Measuring, for every local cost map frame, the distance from the robot's real footprint *outline* to the nearest lethal cell (an exact Euclidean distance transform, not a cost threshold), every finish → start run has its worst moment at the same place, and it is a long moment, not a brush:

run	direction	worst	where (odom)	heading	how long inside 15 cm
5	finish → start	**3.5 cm**	4.40, 4.95	150°	6.4 s
6	finish → start	**2.5 cm**	4.50, 4.89	147°	7.8 s
7	finish → start	**3.5 cm**	4.36, 4.90	151°	5.1 s
9	finish → start	**2.5 cm**	4.57, 4.79	144°	8.1 s
8	start → finish	5.0 cm	3.98, 5.03	–39°	9.6 s
10	start → finish	5.6 cm	2.89, 4.83	–32°	8.7 s

The same junction, in both directions — the headings are nearly opposite — and the two directions are not equally bad. In the four finish → start runs the binding part of the robot is always its **left flank** (the side and hip points, not the nose).



The junction, drawn on one cost map frame, with the robot outline at every frame of each run through it and the tightest pose of each run in orange. Left: the four finish → start runs sweep round the inside of a bend. Right: the two start → finish runs cross the same place almost straight.

It is not the controller

The obvious suspicion is that MPPI cut the corner. It did not. Measuring the robot's lateral offset from the plan it was following, at every cost map frame:

run	2	3	4	5	6	7	8	9
median cross-track (mm)	4	5	3	8	7	6	10	7
p95 cross-track (mm)	22	27	24	45	28	56	47	29

The robot is on its path to within a centimetre, essentially always. And at the tightest frame of each run, the clearance the *plan* had at the point the robot occupied is the same as the clearance the robot had, to within one cost map cell. There is nothing for the controller to have done better.

It is not perception either

The second suspicion is that the obstacle appeared too late in one direction. It did not. Taking the lethal cells that were binding at the moment of closest approach and walking the cost map history backwards to the first frame in which those same cells were already lethal:

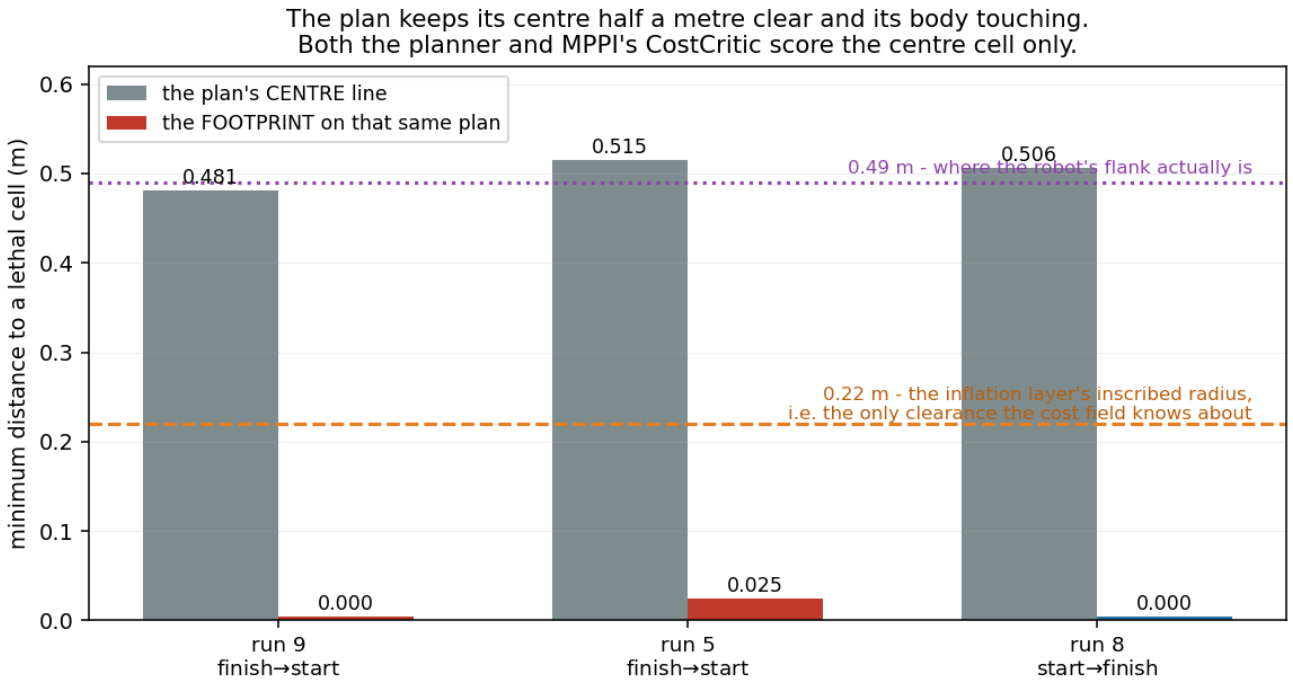
run	direction	first seen	robot's distance then	driven since
5	finish → start	9.7 s before	2.54 m	2.87 m
6	finish → start	11.4 s before	2.71 m	3.61 m
7	finish → start	18.6 s before	4.02 m	4.76 m
9	finish → start	9.2 s before	2.57 m	2.94 m
8	start → finish	10.6 s before	2.61 m	3.19 m
10	start → finish	46.9 s before	3.67 m	10.05 m

Two and a half to four metres of warning, nine seconds or more, in **both** directions. The robot knew.

It is the plan, and the plan is scored on the wrong point

Restricting to the part of each plan that lies inside the junction and scoring it against the cost map that was live at the time, with the real footprint swept along it:

run	plan CENTRE to lethal	FOOTPRINT on that plan	plan points whose centre is inside 0.22 m	plan points whose footprint is inside 0.22 m
9	min 0.481 m, median 0.637	**min 0.000 m**, median 0.106	0.0%	**96.9%**
5	min 0.515 m, median 0.626	**min 0.025 m**, median 0.103	0.0%	**99.3%**
8	min 0.506 m, median 0.594	**min 0.000 m**, median 0.079	0.0%	**99.7%**



The same numbers as a picture. The plan's centre line is held half a metre clear; the robot's flank is 0.49 m from that centre line; the two facts are the same fact.

That shape — the centre comfortably clear, the body touching, and the centre's own clearance sitting almost exactly at the robot's half-width — is the signature of an objective that only ever looks at one cell. It is, and nav2 says so itself. From `nav2_mppi_controller/src/critics/cost_critic.cpp`, in the scoring loop:

```
// The getCost doesn't use orientation
// The footprintCostAtPose will always return "INSCRIBED" if footprint is over it
// So the center point has more information than the footprint
  pose_cost = costmap->getCost(getIndex(x_i, y_i));
...
// Note that we collision check based on the footprint actual,
// but score based on the center-point cost regardless
  traj_cost += pose_cost;
```

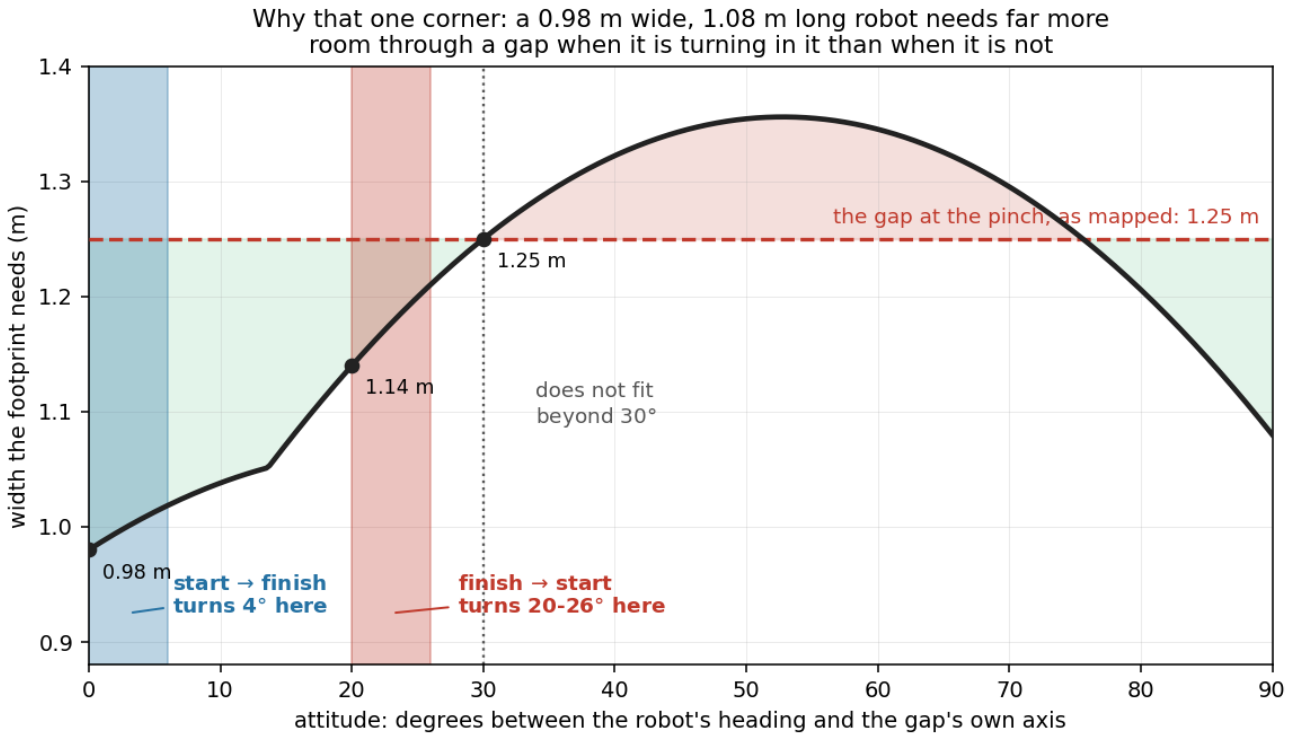
`consider_footprint: true` therefore buys an accurate collision *veto*, not better clearance. The planner is the same: `SmacPlannerLattice` collision-checks the full footprint through `GridCollisionChecker`, but the `cost_penalty` it optimises reads the inflation field at the pose, and the inflation layer builds that field around the **inscribed radius**, which `calculateMinAndMaxDistances` puts at **0.22 m** for this footprint — the front edge.

The flank is 0.49 m out and the tail 0.86 m back. Nothing in the stack has a gradient that knows this.

And the geometry says the answer

None of that yet explains why one direction is twice as bad as the other, because the plan is scored the same way both ways. The answer is in the shape of the gap and the shape of the robot.

At the pinch the free channel is **1.250 m** across at its narrowest, measured over all directions through the binding point. The robot is 0.98 m wide and 1.08 m long. Because those are nearly equal, the width the outline actually needs depends steeply on how obliquely it meets the gap:



The real eight-vertex footprint projected across the pinch's own narrow axis, against the 1.25 m the gap offers. The chamfered tail is why the curve turns over past 50°.

attitude to the gap's axis	width the footprint needs	slack in a 1.25 m gap
0° (square-on)	0.980 m	+0.270 m
10°	1.038 m	+0.212 m
20° — finish → start	**1.140 m**	**+0.110 m**
30°	1.250 m	0.000 m — exactly fills it
40°	1.322 m	–0.072 m — does not fit

And the measured heading change through the junction:

run	5	6	7	9	8	10
direction	F→S	F→S	F→S	F→S	S→F	S→F
turn through the junction	+20°	+26°	+20°	+23°	**+4°**	–29°
worst clearance	3.5 cm	2.5 cm	3.5 cm	2.5 cm	**5.0 cm**	7.1 cm

The answer to the question

That junction is the narrowest thing on the course, and the two directions do not meet it the same way. Going **finish** → **start** the route has to turn 20–26° *while it is inside the gap*, which costs the footprint about 16 cm of extra width and leaves roughly 11 cm of total slack — 5 cm a side before the cost map's own thickness is counted. Going **start** → **finish** the same gap is crossed turning 4°, square-on, with the full 27 cm.

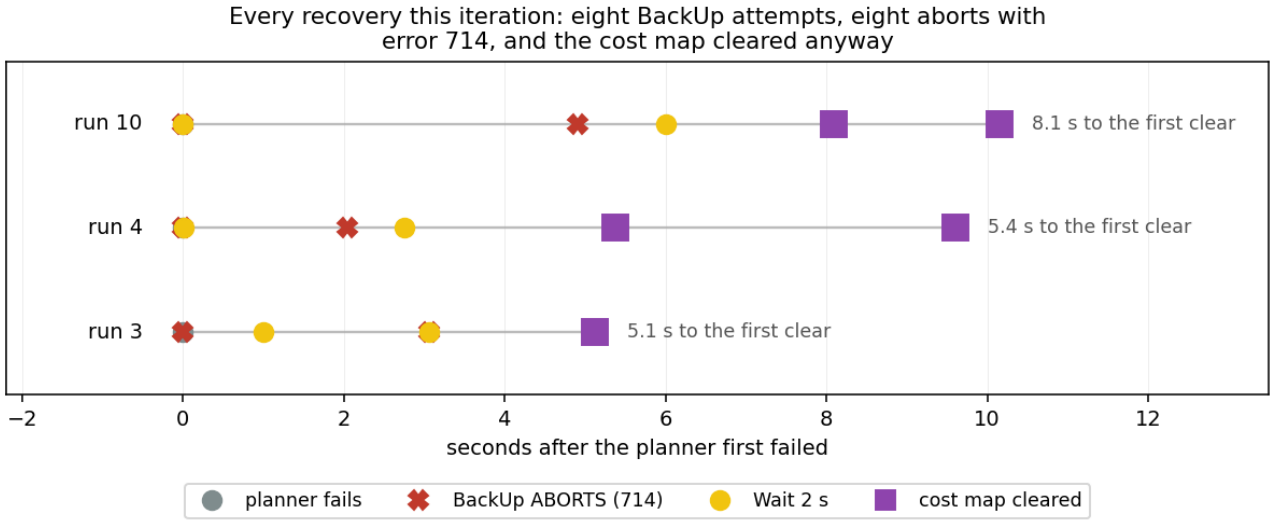
It is not that one direction is driven worse. It is that one direction has to turn where there is no room to turn. Thirty degrees is the hard limit at that gap, and finish → start was running at 20–26.

This also predicts what the team already plans to do next. A course with more turns of this kind will not be harder because the turns are harder — it will be harder wherever a turn is forced to coincide with a narrow point. Turns in the wide sections are free.

3. The recovery branch ran, and every rung under it failed

Iteration 3's change worked. Runs 3, 4 and 10 all failed a plan, all passed the `AreErrorCodesPresent` gate, and all entered the round robin — which is precisely what iteration 3's run 8 could not do. The errors were 208 NO_VALID_PATH this time rather than 205, and the widened list covers both.

What happened next is the problem.



Every recovery episode in the iteration, aligned on the moment the planner first failed.

run	BackUp #1	BackUp #2	first cost map clear	both maps cleared
3	ABORT 714 after 1.00 s	ABORT 714 after 0.00 s	5.1 s in	—
4	ABORT 714 after 0.01 s	ABORT 714 after 0.70 s	5.4 s in	yes, twice
10	ABORT 714 after 0.00 s	ABORT 714 after 1.10 s	8.1 s in	yes

Eight BackUp attempts, eight aborts, every one of them with the same message:

```
[WARN] [behavior_server]: Collision Ahead - Exiting DriveOnHeading
[WARN] [behavior_server]: backup failed:Collision Ahead - Exiting DriveOnHeading
[WARN] [behavior_server]: [backup] [ActionServer] Aborting handle. error_code:714
```

Why BackUp cannot start

From `nav2_behaviors`, `DriveOnHeading::isCollisionFree` simulates the reverse forward by `simulate_ahead_time` at `backup_speed`, breaking early once it has covered the requested distance, and vetoes the **whole manoeuvre** if the footprint at any simulated pose covers a cell of cost 254.

With `simulate_ahead_time: 1.0` and `backup_speed="0.25"` that projection was **0.25 m**. This robot's tail reaches 0.86 m behind `base_link`, and all eight of these aborts happened in places where the measured footprint clearance was 2.5–5 cm. A quarter-metre of straight reverse always put the tail outline on a lethal cell, so the manoeuvre was refused before it began.

> **Why this matters more than the clearance**
>
> The team's standing instruction is that the cost map is cleared only as a last resort. As the tree stood, the two rungs in front of the clear could not run at all, so the clear was reached in five seconds every time — and in runs 4 and 10 the *global* map went too. Run 4's own note in real_report_4 is exactly right: “the robot didn't contact any obstacles when it continued, but that was pure luck.”

4. What was changed

Only `recovery_nav_bt.xml` and `Course2027.yaml`, as agreed. This is a deliberately small change set: section 5 shows the measurements that rule out the obvious alternatives.

recovery_nav_bt.xml — make the recovery able to recover

change	from	to	why
round robin order	BackUp, Wait, BackUp, Wait, clear local, clear both	**Wait**, BackUp, Wait, BackUp, Wait, clear local, clear both	The wait moves to the front. It cannot be refused, it costs nothing, and STVL's in-frustum decay drops a transient mark in about 1 s at `decay_acceleration` 5.0 (measured in iteration 3), so the robot now looks again before it is asked to reverse into anything. It also means six failures are needed to reach a clear instead of five.
`backup_speed` (both)	0.25	**0.10**	The veto window is `backup_speed` × `simulate_ahead_time`, so this takes it from 0.25 m to 0.10 m. The protection is not removed: `onCycleUpdate` re-runs the same footprint check every cycle at 10 Hz, so a reverse still stops before contact — it is simply no longer refused outright before it starts.
first `backup_dist`	0.5	**0.30**	The projection is also capped by the distance still to run, so a shorter first attempt is checked over a shorter span and has the best chance of completing.
second `backup_dist`	0.8	**0.50**	Same reasoning.

A reverse is now slower — 0.30 m takes three seconds instead of 1.2 — and that is accepted. A recovery that takes three seconds and works is better than one that takes ten milliseconds and does not.

Course2027.yaml — one change

parameter	from	to	why
`velocity_smoother.smoothing_frequency`	10.0	**20.0**	Measured on every bag: the header stamp on each `/diffbot_base_controller/cmd_vel` message is 92 ms old at the median and 95–100 ms old whenever the robot is moving — one whole smoother period. The smoother republishes on its own clock and carries its input's stamp, so a fresh MPPI command waits up to 1/f before it is passed on: a mean 50 ms of pure queueing on top of the 40–55 ms of real actuation dead time. At 20 Hz that mean is 25 ms. Nothing about the shaping changes — `feedback` is OPEN_LOOP and `max_accel`/`max_decel` are untouched — and MPPI still runs at 10 Hz.

Nothing else in Course2027.yaml was touched, on purpose. The speed caps, the critic weights, the decay settings and the cost map geometry are all where the team left them. Section 5 is the evidence for leaving them there.

How the changes were verified

what	how	result
The behaviour tree parses and every node resolves	A real `bt_navigator` was started on the edited tree, configured, activated and sent a NavigateToPose goal — in Kilted the XML is built on activation, not on configure, so a configure-only check proves nothing. Control: the same test with `BackUp` renamed to `BackUpBogusXyz`.	**Real tree:** the only error was “Action server compute_path_to_pose not available”, i.e. it got past parsing and registration and stopped only because the planner server was not running. **Control:** “Error at line 135: Node not recognized: BackUpBogusXyz”, which proves registration is checked first and the real tree passed it.
The config loads	Real `velocity_smoother` and `controller_server` started on the edited `Course2027.yaml` and configured.	Both **configure** → **SUCCESS**, no parameter warnings.
The planner experiments in section 5 are trustworthy	The replay harness builds a real global cost map from the bag's LiDAR and tf and then drives a real `planner_server`. Checked against the bag's own published map.	188 lethal cells in the junction against the bag's 184, and the replayed plan reproduced the real one's signature (centre 0.492 m vs 0.481 measured; footprint 0.025 m vs 0.025).

Not verified

Neither change has been run on the robot. The BackUp change is reasoned from the `DriveOnHeading` source and the eight logged aborts, not tested in a real recovery — that needs the course. Watch for a BackUp that actually completes, and be ready to drop `backup_speed` further if 714 still appears.

The smoother change is arithmetic on a measured stamp age, not a closed-loop test. It should show up as that 92 ms median halving in the next bags; if it does not, the delay is somewhere else and the change should go back.

5. What was tried against the real cost map, and rejected

All of the following were run through a real `planner_server` against the global cost map rebuilt from real_bag9's own LiDAR, planning the query the robot actually faced on its way into the junction (`claude_tools/it4_planner_replay.py`). The column that matters is the footprint's minimum clearance, because that is what the team means by nearly hitting something.

configuration	plan centre, min	FOOTPRINT, min	share of the corner under 5 cm	verdict
as shipped (the baseline)	0.506 m	**0.025 m**	14.5–19.1% (four runs)	—
`smooth_path: false`	0.515 m	**0.025 m**	10.6%	The Smac smoother validates each moved point by reading **one cell** and rejecting only cost > 252, so it is blind to this footprint — a real defect, but **not the cause**. Turning it off leaves the minimum exactly where it was.
`cost_penalty` 3 → 8 → 15	0.503 m	**0.025 m**	13.9%, 16.3%	No effect. The term optimises centre clearance and the centre is already where it wants to be.
`cost_scaling_factor` 1.7 → 1.3 → 1.0	0.503 m	**0.025 m**	9.6%, 13.8%, 17.0%	The apparent improvement at 1.0 did not repeat — a second run at the same setting gave 17.0%. The spread is replay noise, not an effect. **Rejected after retesting.**
`footprint_padding` 0.00 → 0.04	—	—	—	**NO_VALID_PATH.** Four centimetres of padding closes the junction entirely. This is the one knob that would align the cost field with the real body, and this course has no room for it.
32-heading lattice (16 → 32)	0.503 m	**0.025 m**	11.2–11.7% on bag 9; **0.0%** → **6.4%** on bag 8	Consistently better in the hard direction and repeatable to the decimal, but it makes the easy direction worse: finer resolution lets the planner find shorter lines, and shorter means closer. **Not adopted.** Planning time was unchanged (100 vs 101 ms).
turning radius 0.5 → 0.8, 0.5 → 1.2	0.480–0.515 m	**0.000–0.025 m**	14.7% / 21.4% on bag 9; 9.0% / 8.4% on bag 8	Worse or inside the noise in every combination. **Not adopted.**

The point of this table

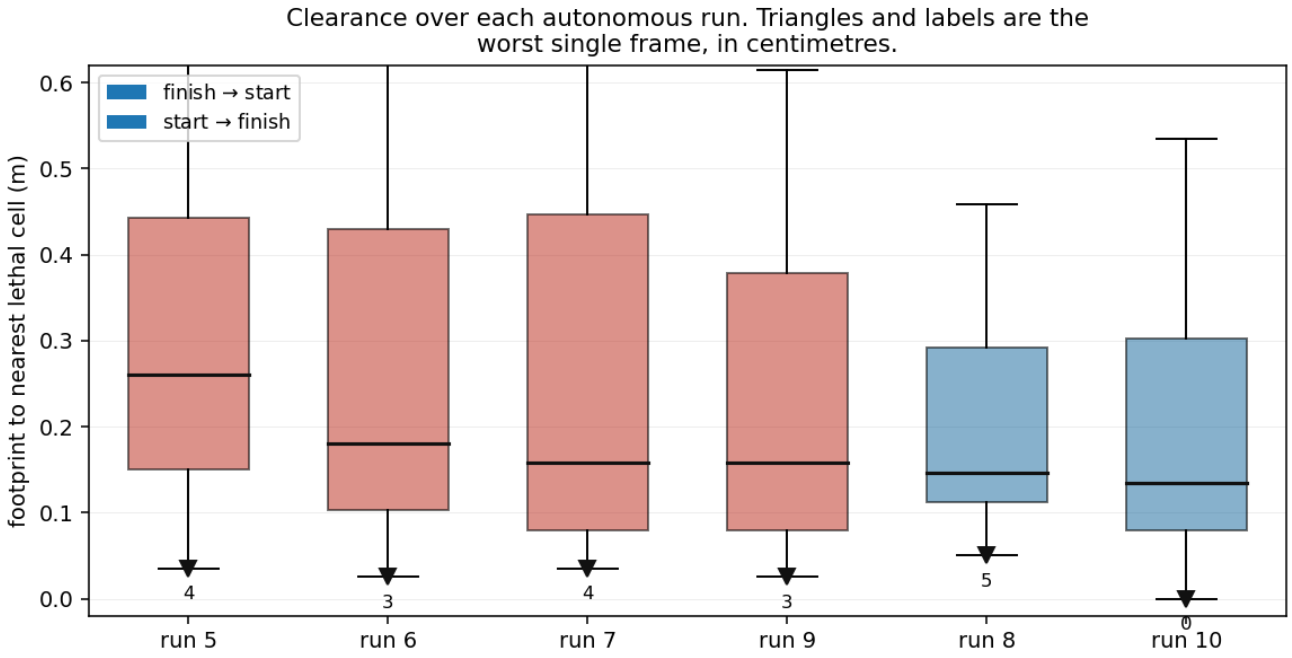
The footprint's minimum clearance at that junction is **0.025 m in every single configuration tested**, including ones that change the planner's objective by a factor of five and ones that change its primitives entirely. That is not a tuning plateau, it is a geometric floor: the gap is 1.25 m, the robot needs 1.14 m at the attitude the route demands, and the cost map's own obstacle thickness eats most of what is left.

No parameter in Course2027.yaml will fix this. The things that will are in section 7.

One hypothesis was also tested and rejected from the bags alone. `CostCritic.near_goal_distance` is 1.5 m, and inside it the critic stops applying its “prefer further from obstacles” term altogether — which looked like a good explanation for the team's note that several runs ended close to a wall. It is not: pooling all autonomous runs, clearance inside 0.6 m of the stopping point has a median of 0.175 m and a worst case of 0.071 m, **better** than the 2.5–4.0 m band (median 0.141 m, worst 0.025 m). The end-of-run clearance is simply where the robot stopped, and it is above the course average. `near_goal_distance` was left alone.

6. Everything else the bags were asked

Clearance, run by run



Boxes are the distribution over each run; the triangle and label are the single worst frame. Run 10's 0 cm is not autonomous — it is the teleoperated drive back out of the lab, and run 1, the teleop baseline not plotted here, reached 0.000 m three times. A human driver brushes this course too.

Speed

Iteration 3's `prune_distance` and critic look-ahead changes did lift the cruise, modestly:

	run 2	run 3	run 5	run 6	run 7	run 8	run 9	run 10
p99 speed while moving (m/s)	0.59	0.72	0.66	0.72	0.68	0.63	0.67	0.64
peak (m/s)	0.62	0.74	0.67	0.72	0.69	0.64	0.68	0.67

Against iteration 3's 0.50 m/s (run 7) and 0.63 m/s (run 9), that is a real if unspectacular gain, and nothing came close to the 5 mph limit — the velocity smoother's 2.235 m/s cap was never approached. Still 0% of moving samples reached 0.8 of the `vx_max` 1.0 cap.

No speed change was made this iteration and that is deliberate. The binding problem is a 1.25 m gap crossed with centimetres to spare; going faster through it trades directly against the thing the team asked to improve. 0.6–0.7 m/s through a slalom this tight is reasonable, and the speed work should resume once the clearance has somewhere to come from.

Loop timing

Every control loop held its deadline, again:

loop	configured	measured median	p99	late cycles
controller / cmd_vel out	10 Hz	100 ms	100–101 ms	0%
planner (BT RateController)	2 Hz	541–566 ms	591–746 ms	0%
local cost map publish	10 Hz	**194–199 ms**	201–241 ms	— see below

The local cost map is configured to publish at 10 Hz and is delivering 5. That is the price of iteration 3's `always_send_full_costmap: true` on a 400 × 400 grid. It does **not** affect control — the map lives in `controller_server`'s own process and MPPI reads it directly, and the controller held exactly 100 ms throughout — it only limits what can be seen from the bag afterwards. It has been left on, because this iteration's whole clearance analysis depended on it. Turn it off for a competition run.

The yaw scale, and why the cost map is fat

The wheel-separation correction landed and did most of the job. Regressing the IMU's gyro against the wheel-derived yaw rate, over all ten bags:

	iteration 3	iteration 4
gyro / wheel yaw rate	1.238–1.262	**1.0619 ± 0.0049**
implied effective track	~0.816 m (config 1.016)	0.806 m (config **0.856**)
residual error	~25%	**6.2%**

Unlike iteration 3's number, this one comes from ten course runs full of arcs rather than in-place spins, so it is much less contaminated by scrub — and it is consistent with the 1.0514 that latency_claude_3's step tests gave independently. A 6% yaw-scale error means the robot's heading estimate drifts, and a drifting heading paints every LiDAR return at a slightly wrong bearing. Measured on the local cost map at the tightest moment of runs 8 and 9, the apparent thickness of obstacles is:

	median	p90	max
apparent obstacle thickness in the local cost map	0.050–0.071 m	0.112–0.141 m	0.224–0.250 m

A flat wall and the flat side of a bin should be one or two 2.5 cm cells thick. Five to fourteen centimetres is smear, and it is subtracted from the free channel on **both** sides of every gap. At a pinch with 11 cm of slack, that is most of the budget. This is the single largest recoverable source of clearance on the robot and it costs nothing to fix in software — see section 7.

The oscillation around the third and fourth bins

Nothing in the control chain is hunting. Fitting a pure delay between the commanded and measured yaw rate and looking at what is left over:

	run 2	run 3	run 5	run 6	run 7	run 8	run 9
best-fit lag (ms)	290	300	300	250	300	300	270
residual sd (rad/s)	0.027	0.038	0.025	0.024	0.028	0.027	0.026

A residual of 0.025–0.038 rad/s against commands with a standard deviation of 0.20–0.23 is tight tracking; the encoder's own quantisation is 0.0109 rad/s of yaw, so roughly half of that residual is the sensor. The motor noise the team heard is below what the 100 Hz odometry can resolve — it is inside board B's PID loop, and the right instrument for it is the step test, not a course bag.

A loose end in the command chain

During every recovery, `diff_drive_controller` logged:

```
[WARN] Velocity command timed out. Braking.
[WARN] Ignoring the received message (timestamp 1793752872.8889701366) because it is
  older than the current time by 0.5359343710 seconds
```

The timestamp is **frozen** while the clock advances. After the behaviour server stopped commanding, the velocity smoother kept republishing its last zero command at 10 Hz with that command's original stamp until `velocity_timeout` expired a second later, and `diff_drive_controller` rejected each one as stale. The outcome is safe — the controller had already braked on timeout — but it confirms the smoother does not restamp its output, which is the same fact behind the 92 ms median stamp age that section 4's change addresses. Worst observed stamp age was 995 ms, against a tolerance of 500 ms.

7. Recommendations outside the two files

Flagged, not changed, as agreed. These are ordered by how much clearance they are worth.

priority	where	what
1	`imprimis_hardware_platform/config/diffbot_controllers.yaml`, `wheel_separation: 0.856`	A 6.2% residual yaw-scale error remains, consistent across all ten runs and consistent with the independent step-test figure of 1.0514. The implied track is about **0.806 m**. This is now the **largest single source of lost clearance**: the heading drift it causes is what paints obstacles 5–14 cm thick, and at a pinch with 11 cm of slack that is most of the budget. Before changing the number, measure the track with a tape — and note that a figure-of-eight or a constant-radius arc is a far better calibration run than an in-place spin, which scrubs hardest.
2	`imprimis_hardware_platform/launch/imprimis_hardware.launch.py`	The `update_rate: 60` edit is in the source tree but not in `install/`, so the IMU ran at 20 Hz for all ten bags. `colcon build --packages-select imprimis_hardware_platform` and relaunch. Worth about 17 ms of the IMU's remaining 24.8 ms.
3	the course itself	The junction is **1.25 m** wide as mapped and the route through it needs 1.14 m. If that corner can be opened by even 15 cm, `footprint_padding` becomes usable — and padding is the *only* knob that makes the cost field describe this robot's real body rather than a 0.22 m disc. Today it is not usable: 4 cm of it returns NO_VALID_PATH.
4	`src/um7` (vendored)	Still open from latency_claude_3. `BAUD_115200` is hard-coded into CREG_COM_SETTINGS regardless of the `baud` parameter, which caps the broadcast at 75.8 Hz for this register set. 60 Hz is the practical maximum without patching it, which is why the launch file asks for 60 and not 100.
5	`imprimis_perception` — `surface_tools.classify_lidar`, `surface_max_height: 0.60`	Open since iteration 2 and still worth closing. `/velodyne_points_nav` contains nothing below 0.60 m above the ground, so the cost map only ever learns the top of an object.
6	nav2, structurally — no file to edit	Neither the planner nor MPPI has any footprint-clearance *gradient*; both score the centre cell and use the footprint only as a veto (`cost_critic.cpp` says so in a comment). For a robot with a 0.22 m inscribed radius and a 0.49 m half-width this is systematically optimistic by up to 0.27 m. If clearance ever has to be guaranteed rather than hoped for, the honest answer is a custom MPPI critic that scores the swept outline's distance to the nearest lethal cell. That is a real piece of work and should not be started before the calibration in row 1 is done, because that is cheaper and probably sufficient.
7	the bag command	`/speed_governor_info` should be `/speed_governor/info` — it recorded zero messages again, third iteration running. Also worth adding `/local_costmap/costmap_raw` and `/global_costmap/costmap_raw`, which carry true 0–255 costs instead of the 0–100 rescale this analysis had to invert.
8	`imprimis_hardware_platform/hardware/diffbot_system.cpp`	Still open from latency_claude_3: `read()` silently reuses the previous velocity on a failed read with no dropped-read counter, and `write()`'s failure branch is an empty block. Neither showed up in these bags, but neither would.

8. What to look for in the next run

1. **Does a BackUp ever complete?** The behaviour-tree log and `behavior_server`'s own output answer this directly. Success is a `BackUp` going RUNNING → SUCCESS, or at worst aborting *after* covering some distance instead of in 10 ms. If 714 still appears immediately, drop `backup_speed` to 0.06 and say so — the mechanism is understood, only the number is uncertain.

2. **Is the cost map still being cleared?** Count `Received request to clear entirely` in `/rosout`. This iteration it appeared five times across three runs, and three of those were the global map. Zero is the target; anything reached only after the new wait and both reverses have genuinely been tried is acceptable.

3. **Did the cmd_vel stamp age halve?** It should go from a 92 ms median to about 46. If it does not, the delay is not in the smoother and the change should be reverted.

4. **If the track width gets measured and corrected**, re-measure the obstacle thickness before anything else. The prediction is that it drops towards one or two cells, and that the free channel at the junction widens by something like 5–10 cm. That is the result that would make `footprint_padding` worth retesting, and it is the one number that would change this report's conclusion.

5. **For the harder course that is planned next**, the useful thing to vary is not how sharp the turns are but whether a turn coincides with a narrow point. A 25° turn in a 2 m corridor is free; the same turn in a 1.25 m gap is what this iteration was about. If the new course can include one turn of each kind at a measured width, the attitude curve in section 2 can be checked against reality rather than against the cost map.

Tools written for this iteration are in `claude_tools/`: `it4_cache.py`, `it4_freshness.py`, `it4_clearance.py`, `it4_corners.py`, `it4_events.py`, `it4_plan_vs_driven.py`, `it4_plan_quality.py`, `it4_plan_corner.py`, `it4_plan_decay.py`, `it4_warning.py`, `it4_throat.py`, `it4_map_fidelity.py`, `it4_moment.py`, `it4_course.py`, `it4_corner_compare.py` and `it4_planner_replay.py`, which is the one that drives a real planner against a cost map rebuilt from a bag.